

ТЕОРИЯ ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЕЙ

Каганов Ю.Т.

к.т.н., доцент каф. ИУ-9

**«Теоретическая информатика и компьютерные
технологии»**

ЛЕКЦИЯ № 5

Математические основы ТИНС

Часть 2

ПЛАН ЛЕКЦИИ

- 1. Методы многомерной оптимизации с ограничениями.**
- 2. Классификация типов задач многомерной оптимизации.**
- 3. Классификация типов методов решения задач оптимизации.**
- 4. Градиентные методы в машинном обучении.**
- 5. Стохастический градиент.**
- 6. Современные методы многомерной оптимизации ориентированные на использование в искусственных нейронных сетях :**
 - 1. NAG. (Nesterov accelerated gradient)**
 - 2. Adagrad. (Adapted gradient)**
 - 3. Adadelata. (Extension of Adagrad)**
 - 4. RMSProp. (Adaptive Root Mean Squared)**
 - 5. Adam. (Adaptive moment estimation)**

Необходимые и достаточные условия существования экстремума функции

Условия существования экстремума с учетом ограничений

Постановка задачи:

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$ $y=f(x)$ – выпуклая функция;

$x \in X, y \in Y; f(x) \in C^1(X)$ и пусть $g: X \rightarrow Z; X \subset E^n, Z \subset E^m$
 $f(x) \in C^1(X)$

$g_i(x) \in C^1(X), j = 1..m.$ – вогнутые или выпуклые функции,

$g = [g_1(x), g_2(x), \dots, g_m(x)]^T$, а также $\forall g_j(x) = 0$ – ограничения типа равенства.

Тогда задачу поиска экстремума функции при наличии ограничений можно записать:

$$\begin{cases} x^e = \text{Arg min } f(x) \\ x_i \geq 0, i = 1..n \\ g_j(x) = 0, j = 1..m \end{cases}$$

Необходимые и достаточные условия существования экстремума функции.

Метод множителей Лагранжа.

(1788г. "Аналитическая механика")

Функция вида: $L(x, \lambda) = f(x) + \sum_{j=1}^m \lambda_j g_j(x).$

Если взять вариацию от функции $L(x, \lambda)$ в $U_\varepsilon(x^e)$

$$\delta L(x, \lambda) = \sum_{i=1}^n \frac{\partial f(x)}{\partial x_i} \delta x_i + \sum_{i=1}^n \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} \delta x_i = 0 \quad \text{при} \quad x = x^e$$

$$\delta L(x, \lambda) = \sum_{i=1}^n \left[\frac{\partial f}{\partial x_i} + \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} \right]_{x=x^e} \delta x_i = 0$$

Таким образом получаем систему уравнений :

$$\left[\frac{\partial f}{\partial x_i} + \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} = 0 \right]_{x=x^e}$$

Подбором определенных значений λ_j можно добиться того, чтобы эти уравнения удовлетворялись при $\forall i = \overline{1..n}$. Именно в точке x^e достигается стационарность функции $L(x, \lambda)$, т.е. экстремум функции $f(x)$ при заданных ограничениях $g_j(x)=0$.

Необходимые и достаточные условия существования экстремума функции.

Метод множителей Лагранжа.

Задачу можно переформулировать следующим образом.

Требуется найти:

$$\begin{cases} x^e = \text{Arg min}_{x \in X} L(x, \lambda) \\ \lambda^e = \text{Arg max}_{\lambda \in \Lambda} L(x, \lambda) \end{cases}$$

Тогда решение задачи состоит в совместном решении уравнений:

$$\begin{cases} \frac{\partial L(x, \lambda)}{\partial x_i} = 0, i = 1..n \\ \frac{\partial L(x, \lambda)}{\partial \lambda_j} = 0, j = 1..m \end{cases} \quad \begin{cases} \frac{\partial f}{\partial x_i} + \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} = 0, -n \text{ _ уравнений} \\ g_j(x) = 0, -m \text{ _ уравнений} \end{cases}$$

Составляются и решаются 2 системы уравнений:

Необходимые и достаточные условия существования экстремума функции.

Область применения классического метода множителей Лагранжа.

Задача поиска экстремума методами классического математического анализа связана с удовлетворением следующих условий, накладываемых на функции:

1. Целевая функция $f(x)$ должна быть выпуклой (в случае поиска минимума).
2. Функции ограничений $g_j(x)$ должны быть вогнутыми или выпуклыми функциями.
3. Ограничения $g_j(x) = 0$ только типа равенств.

Последнее условие связано с тем, что если $g_j(x) < 0$, то классический метод множителей Лагранжа не дает единственного решения $x^e = \text{Arg min } f(x)$ и, следовательно, не работает.

Задачи выпуклого программирования

Формулировка задачи

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$,

$g: X \rightarrow Z; X \subset E^n, Z \subset E^m$,

$f(x) \in C^1(X), g(x) \in C^1(X)$,

$x^e = \operatorname{Arg} \min f(x), x_i \geq 0, i = 1..n$

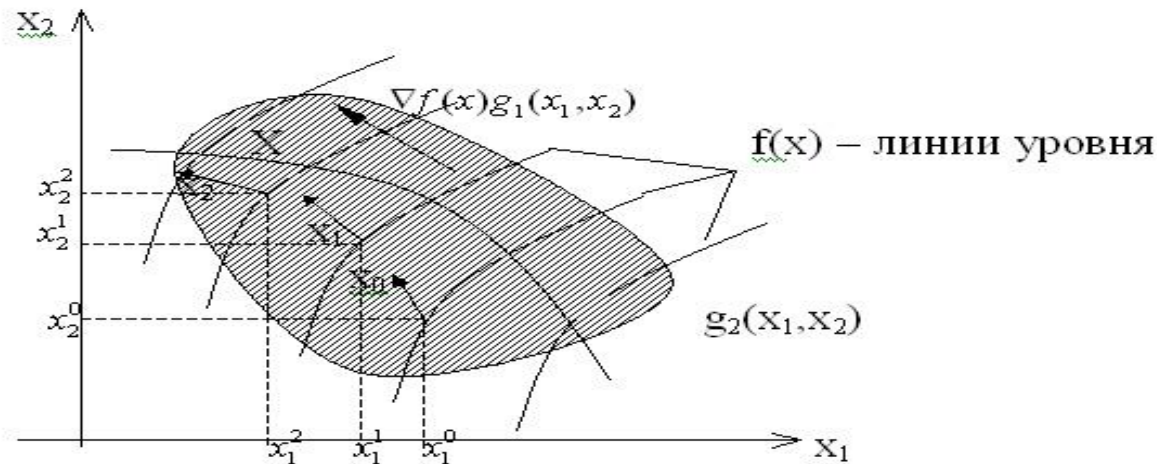
$g_j(x) = 0, j = 1..\ell$ – ограничения типа равенств.

$g_j(x) \leq 0, k = (\ell + 1)..m$ – ограничения типа неравенств.

Обозначим множество индексов: $K = \{l + 1, m\}, k \in K$.

Задачи выпуклого программирования.

Активные ограничения.



В процессе решения задачи происходит перемещение текущей точки из заданной начальной точки в конечную точку. При этом возможен выход на границу области допустимых решений X . Такие нарушенные ограничения носят название активных.

Определение множества индексов активных ограничений:

$$K_A = \left\{ k \in K \mid (\forall g_k(x) \leq 0) \wedge (\exists g_{k_A}(x) = 0) \Rightarrow k_A \in K_A \subset K \right\}$$

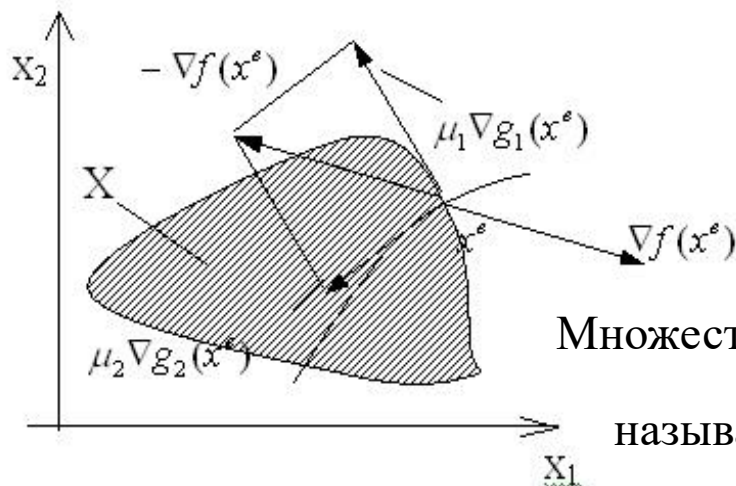
Функция Куна-Таккера (функция Лагранжа) для задач выпуклого программирования (1951г.)

$$L(x, \lambda, \mu) = f(x) + \sum_{j=1}^l \lambda_j g_j(x) + \sum_{k=l+1}^m \mu_k g_k(x)$$

Продифференцируем функцию $L(x, \lambda, \mu)$ по x и приравняем 0.

Если продифференцировать по μ_k , то получим неравенства $g_k(x) \leq 0$, что не позволяет получить единственное решение. Противоречие состоит в том, что для ЗВП имеется единственное решение.

$$\left. \frac{\partial L(x, \lambda, \mu)}{\partial x_i} \right|_{x^e} = 0 \Rightarrow \nabla_x f(x^e) = -\sum_{j=1}^l \lambda_j \nabla g_j(x) - \sum_{k=l+1}^m \mu_k \nabla g_k(x)$$



Лемма: Градиент целевой функции $\nabla f(x^e)$ равен линейной комбинации градиентов функций ограничений в точке условного экстремума x^e : $-\nabla f(x^e) = \sum_{j=1}^l \lambda_j \nabla g_j(x^e) + \sum_{k=l+1}^m \mu_k \nabla g_k(x^e)$

Множество всех векторов

называется множеством **возможных направлений**.

Теорема Каруша-Куна-Таккера

Рассмотрим задачу выпуклого программирования

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1; f(x) \in C^1(X)$

$g: X \rightarrow Z; X \subset E^n, Z \subset E^m; g(x) \in C^1(X)$

Требуется найти: $x^e = \text{Arg min } f(x), x \in X$

$x_i \geq 0, i = 1..n$,

$g_j(x) = 0, j = 1..l$,

$g_k(x) \leq 0, k = l + 1..m.$ Для этого решение ЗВП должно удовлетворять следующим условиям:

$x^e \in X,$

$\mu_k g_k(x^e) = 0, \lambda_j g_j(x^e) = 0.$

$\nabla_x f(x^e) + \sum_{j=1}^l \lambda_j \nabla g_j(x^e) + \sum_{k=l+1}^m \mu_k \nabla g_k(x^e) = 0$. Условия:

$\mu_k \begin{cases} = 0, g_k(x) < 0, \\ > 0, g_{kA}(x) = 0. \end{cases}$

Если эти условия выполнены, то

x^e – решение ЗВП.

- называются **условиями дополняющей нежесткости.**

Теория двойственности

Седловая точка функции Лагранжа. Двойственность.

Рассмотрим упрощенную задачу выпуклого программирования.

Пусть заданы условия на $f(x)$ и $g(x)$ как и ранее.

$$\begin{cases} x^e = \text{Arg min } f(x), \\ x_i \geq 0, i = 1..n, \\ g_j(x) \leq 0, j = 1..m. \end{cases}$$

Тогда функция Лагранжа может быть записана:

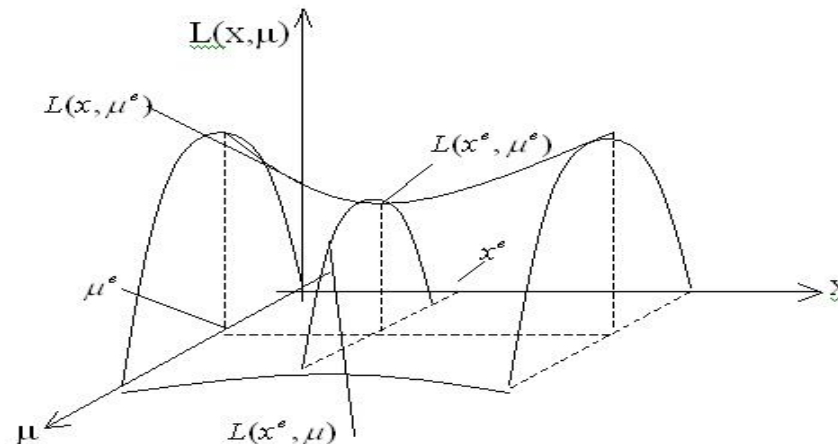
$$L(x, \mu) = f(x) + \sum_{j=1}^m \mu_j g_j(x).$$

Рассмотрим случай, когда $x = x^e, \mu = \mu^e$:

$$L(x, \mu^e) \geq L(x^e, \mu^e) \geq L(x^e, \mu).$$

Таким образом седловая точка функции Лагранжа (функции Куна-Таккера) является $[x^e, \mu^e]^T$.

Именно эта точка является решением ЗВП.



Постановка задачи многомерной оптимизации

Рассмотрим задачи нелинейной оптимизации в общем случае.

Пусть $x = (x^{(1)}, \dots, x^{(n)})^T \in R^n$, S – некоторое множество из пространства R^n . Пусть $f_0(x), \dots, f_m(x): S \rightarrow R^1$. Тогда задачу минимизации можно рассматривать в следующем виде

$$\min f_0(x) \text{ при } f_j(x) \leq 0, j = 1, \dots, m, x \in S. \quad (1)$$

$f_0(x)$ - будем называть *целевой функцией*. Функции $f_j(x) \leq 0$, как было принято нами ранее, обозначим $g_j(x) \leq 0$ или $g_j(x) = 0$.

Они называются *функциями ограничений* и зависят от типа ограничений – равенств или неравенств. Множество S –

называется *базовым допустимым множеством*, а множество Q

$Q = (x \in S | g_j(x) \leq 0, j=1, \dots, m)$. Называется просто *допустимым множеством*.

Классификация типов задач многомерной ОПТИМИЗАЦИИ

- *Задачи на условный экстремум: $Q \subset R^n$.*
- *Задачи на безусловный экстремум: $Q \equiv R^n$.*
- *Гладкие задачи: $\forall f_j(x) \in C^k(Q), j = 0, \dots, m$.*
- *Негладкие задачи: $\exists f_j(x) \notin C^k(Q), j = 0, \dots, m; \forall k \geq 1$.*
- *Задачи с линейными ограничениями: все функциональные ограничения являются линейными функциями:*

$$f_j(x) = \sum_{i=1}^n a_j^i x^i + b_j, j = 1, \dots, m. \text{ При этом}$$

*базовое множество S является многогранником. Если $f_0(x)$ также линейная функция, то исходная задача (1) называется **задачей линейной оптимизации**.*

Классификация типов задач многомерной оптимизации

1. Свойства допустимых множеств.

- Задача (1) называется *допустимой*, если $Q \neq \emptyset$.
- Задача (1) называется *строго допустимой*, если существует такой вектор $x \in \text{int}Q$, что $f_j(x) < 0$ (или > 0) для всех ограничений-неравенств и $f_j(x) = 0$ для всех ограничений-равенств (условие Слэйтера).

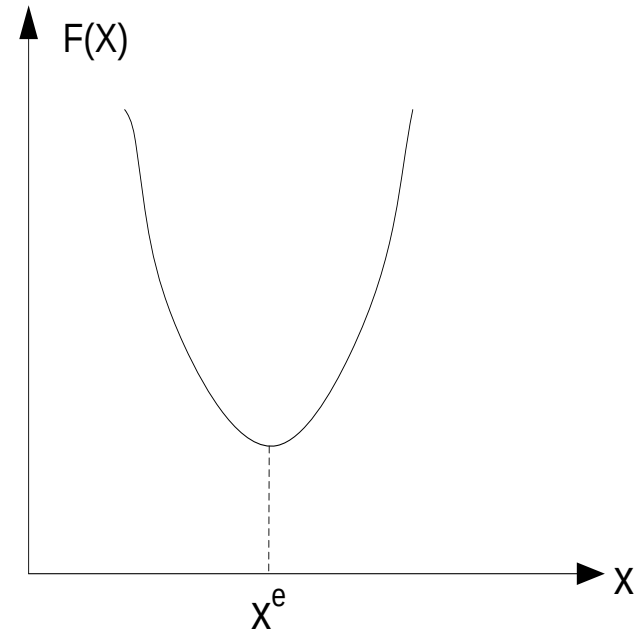
2. Типы решений задачи (1).

- Точка x^* называется оптимальным глобальным решением задачи (1), если $f_0(x^*) \leq f_0(x)$ для всех $x \in Q$ (*глобальный минимум*). В этом случае $f_0(x^*)$ называется (*глобальным*) оптимальным значением задачи.
- точка x^* называется *локальным решением задачи (1)*, если для всех $x \in \bar{Q} \subset Q$ выполнено неравенство $f_0(x^*) \leq f_0(x)$ (*локальный минимум*).

Один экстремум

Виды задач:

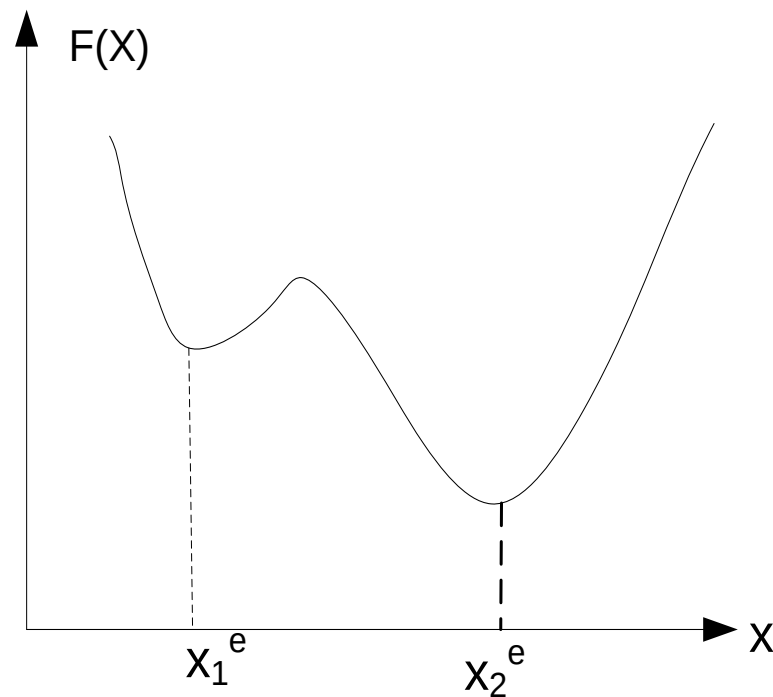
- методы **одноэкстремальной** оптимизации (единственный экстремум – унимодальная функция);



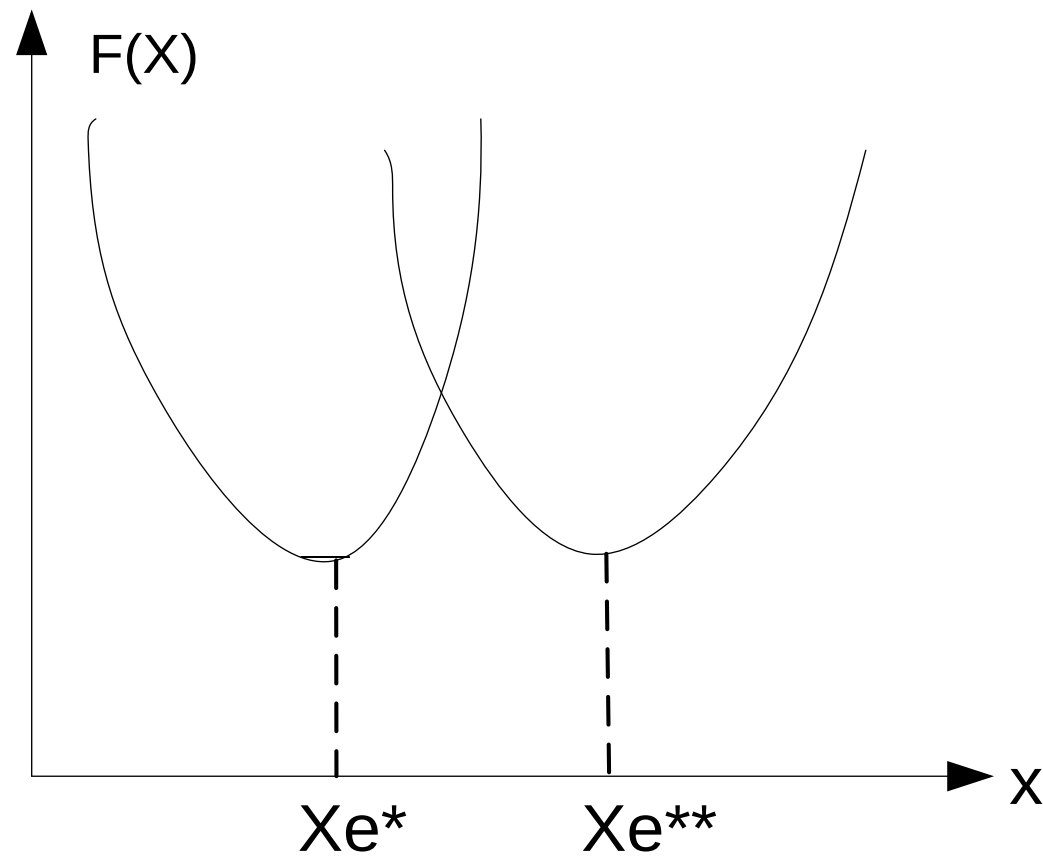
Много экстремумов

Виды задач:

методы многоэкстремальной
оптимизации (много
экстремумов — полимодальная
функция);

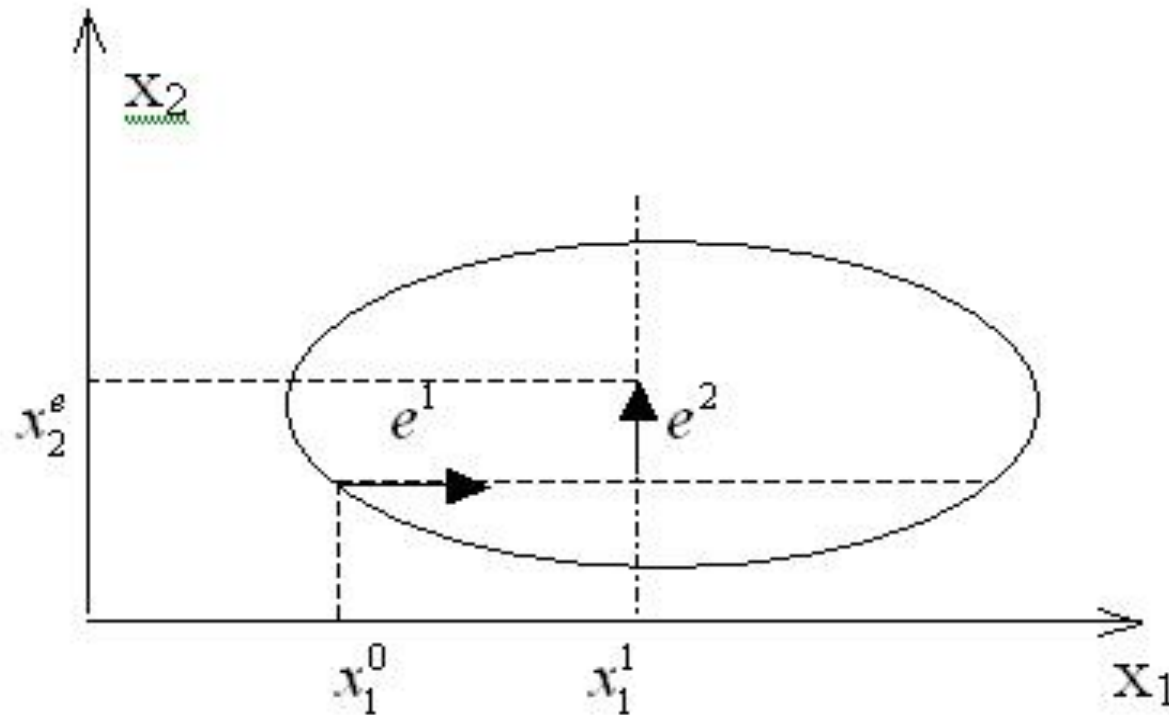


Многокритериальная оптимизация



Метод покоординатного спуска (метод 0-го порядка)

Функция $\varphi(x) \in C^0(X)$ – непрерывная функция.



Алгоритм:

Ш1. Направление d^k выбирается вдоль координаты x_k от точки $x^{(k)}$, т.е.

$$d^{(k)} = \frac{x_k}{\|x_k\|} = e^k$$

Ш2. Определяется величина шага

$$\alpha^{*(k)} = \operatorname{Arg} \min_{\alpha \in E^1} \varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$$

Ш3. Формируется новое значение

$$x^{(k+1)} = x^{(k)} + \alpha^{*(k)} d^{(k)}$$

Ш4. Проверка сходимости:

Если $(|\varphi(x^{(k+1)}) - \varphi(x^{(k)})| \leq \varepsilon, \|x^{(k+1)} - x^{(k)}\| \leq \delta)$, то *КОНЕЦ*

Иначе $k=k+1$ и переход на Ш 1.

Достоинства: возможность оптимизации для задач с недифференцируемыми функциями.

Недостатки:

1. Низкая скорость сходимости (сублинейная, т.е. $\gamma < 1$)

2. Возможность закливания при плохой обусловленности задачи, т.е. когда функция $\varphi(x)$ имеет овражный характер.

Обусловленность задачи

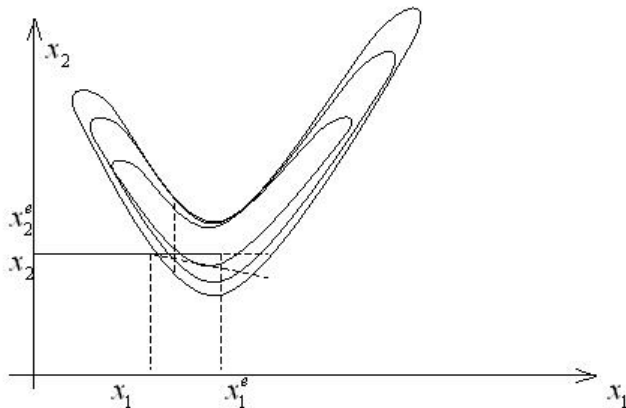
Определяется числом обусловленности матрицы Гессе (если она существует), т.е. отношением $\chi = \frac{L}{l}$.

Где L - максимальное собственное число $H(x^{(k)})$

l – минимальное собственное число $H(x^{(k)})$.

В случае отсутствия матрицы Гессе числом обусловленности может быть отношение:

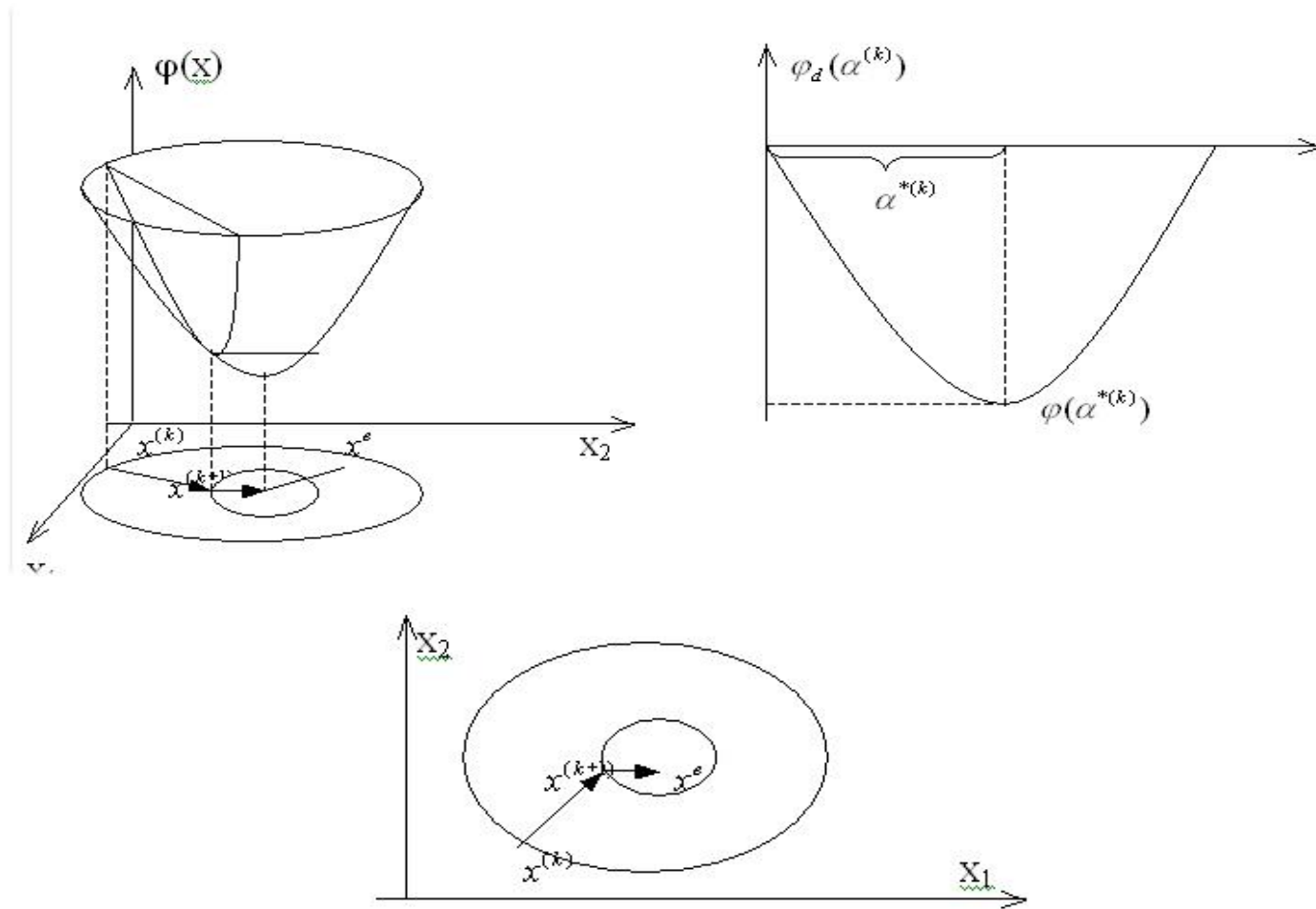
$$\chi = \lim_{\delta \rightarrow 0} \frac{\sup \|x^{(k)} - x^e\|^2}{\inf \|x^{(k)} - x^e\|^2} .$$



Пример функция Розенброка :

$$f(x_1, x_2) = 100(x_2^2 - x_1)^2 + (x_2 - 1)^2$$

Метод наискорейшего спуска (метод 1-го порядка).



Метод наискорейшего спуска (метод 1-го порядка).

Функция $\varphi(x) \in C^1(X)$ - класс 1 раз дифференцируемых функций.

$x^{(k+1)} = x^{(k)} + \Delta x^{(k)} = x^{(k)} + \alpha^{(k)} d^{(k)}$ – в ряд Тейлора, тогда, если учитывать
Разложим функцию $\varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$ только 2 члена ряда, получим:

$$\varphi(x^{(k+1)}) = \varphi(x^{(k)}) + \nabla^T \varphi(x^{(k)}) \Delta x^{(k)} + o(\|\Delta x^{(k)}\|)$$

Для того, чтобы функция уменьшалась, необходимо, чтобы $d^{(k)} \sim -\nabla \varphi(x^{(k)})$
 $d^{(k)} = -\frac{\nabla \varphi(x^{(k)})}{\|\nabla \varphi(x^{(k)})\|}$. Тогда $x^{(k+1)} = x^{(k)} - \frac{\nabla \varphi(x^{(k)})}{\|\nabla \varphi(x^{(k)})\|} \alpha^{*(k)}$

При этом $\alpha^{*(k)} = \text{Arg min}_{\alpha \in E^1} \varphi(x^{(k)} + \alpha^{*(k)} d^{(k)})$

Особенность МНС: $\nabla^T \varphi(x^{(k+1)}) \nabla \varphi(x^{(k)}) = 0$; то есть $d^{(k+1)T} d^{(k)} = 0$.

Достоинства. Линейная скорость сходимости для функций $\varphi(x)$ близких к квадратичным.

Недостатки:

1. Функция $\varphi(x) \in C^1(X)$ – должна быть дифференцируема.
2. Возможность закливания при плохой обусловленности задачи.

Метод Ньютона (метод второго порядка)

Условием использования этого метода является:

$$\varphi(x) \in C^2(X),$$

а также условие выпуклости $\varphi(x)$ в окрестности точки поиска.

Точка x^* — стационарная точка (точка локального экстремума) .

Пусть: $\varphi(x^*) = \min(\varphi(x))$ Тогда :

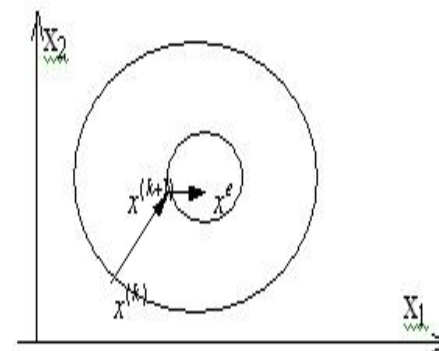
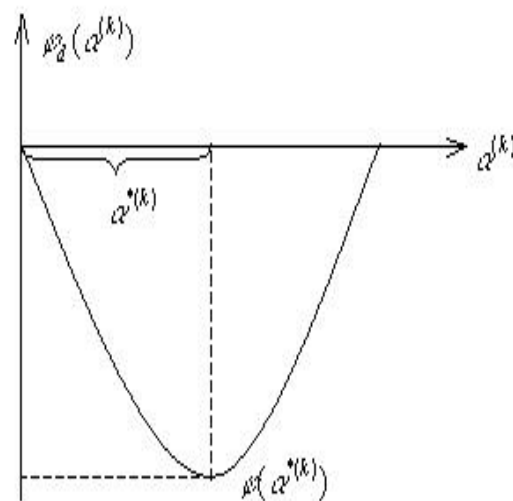
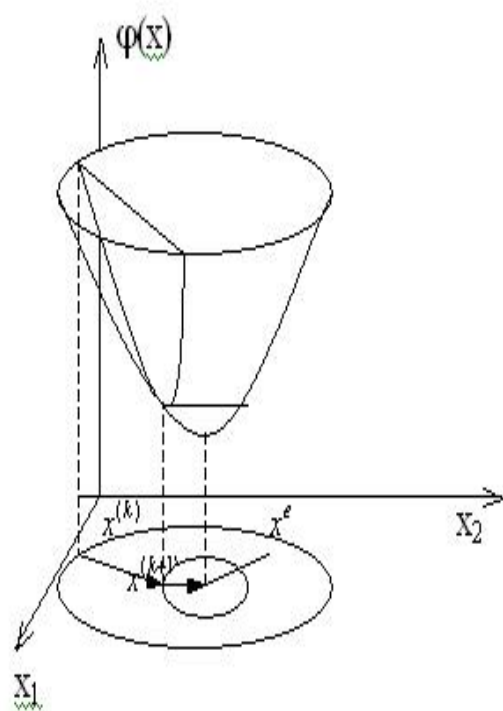
$$\bar{x} \in U_\varepsilon(x^*) \quad \varphi(x^*) = \varphi(\bar{x} + \Delta x) = \varphi(\bar{x}) + \nabla^T \varphi(x) \Delta x^* + \frac{1}{2} \Delta x^T H(\bar{x}) \Delta x + o(\|\Delta x\|^2)$$
$$x^* = \bar{x} + \Delta x \quad \text{где} \quad H(\bar{x}) = \left\{ \frac{\partial^2 \varphi(\bar{x})}{\partial x_i \partial x_j} \right\}_{i=1..n, j=1..n}$$

Разложение функции $\varphi(x^*)$ в окрестности x^* в ряд Тейлора до 3-го члена дает квадратичную аппроксимацию этой функции в $U_\varepsilon(x^*)$:

$$0 = \nabla \varphi(\bar{x}) + H^T(\bar{x})(x^* - \bar{x}) \quad . \quad x^{(k+1)} = x^{(k)} - H^{-1}(x^{(k)}) \nabla \varphi(x^{(k)}).$$

$$d^{(k)} = -H^{-1}(x^{(k)}) \nabla \varphi(x^{(k)}); \quad \alpha^{*(k)} = \text{Arg min } \varphi_d(\alpha^{(k)}); \quad x^{(k+1)} = x^{(k)} + \alpha^{*(k)} d^{(k)} .$$

Метод Ньютона (метод второго порядка)



Метод Ньютона (метод второго порядка).

Сопряженность: $d^{(k+1)} H^{-1}(x^{(k)}) d^{(k)} = 0$
Достоинства :

Для функций, близких к квадратичным, скорость сходимости алгоритма суперлинейная.

Недостатки:

- 1.Необходимость вычисления $H^{-1}(x^{(k)})$.
- 2.Процесс решения сильно зависит от выбора $x^{(0)}$ (т.е. от начальной точки)
- 3.Функция $\varphi(x)$ должна быть выпуклой во всех точках $x^{(k)}$, то есть $\det H(x^k) > 0$.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ

За последние 18-20 лет произошел огромный скачок в развитии систем искусственного интеллекта на основе машинного обучения и глубоких нейронных сетей. Это было обусловлено четырьмя основными факторами:

1. Значительным повышением производительности компьютерных систем.
2. Наличием доступных очень больших баз данных.
3. Созданием новых методов оптимизации на основе стохастического градиентного спуска.
4. Разработкой и созданием систем параллельной обработки информации на основе CUDA, GPU и TPU.

Мы рассмотрим те методы оптимизации, которые легли в основу машинного обучения и теории глубоких нейронных сетей.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ

Метод градиентного спуска является одним из наиболее популярных алгоритмов в области оптимизации для реализации обучения в современных нейронных сетях. В настоящее время разработан целый ряд библиотек глубокого машинного обучения, реализованных в виде соответствующих фреймворков. Ядром этих фреймворков являются методы оптимизации, основанные на использовании различных вариантов стохастического градиентного спуска.

Фреймворки для работы с глубокими сетями

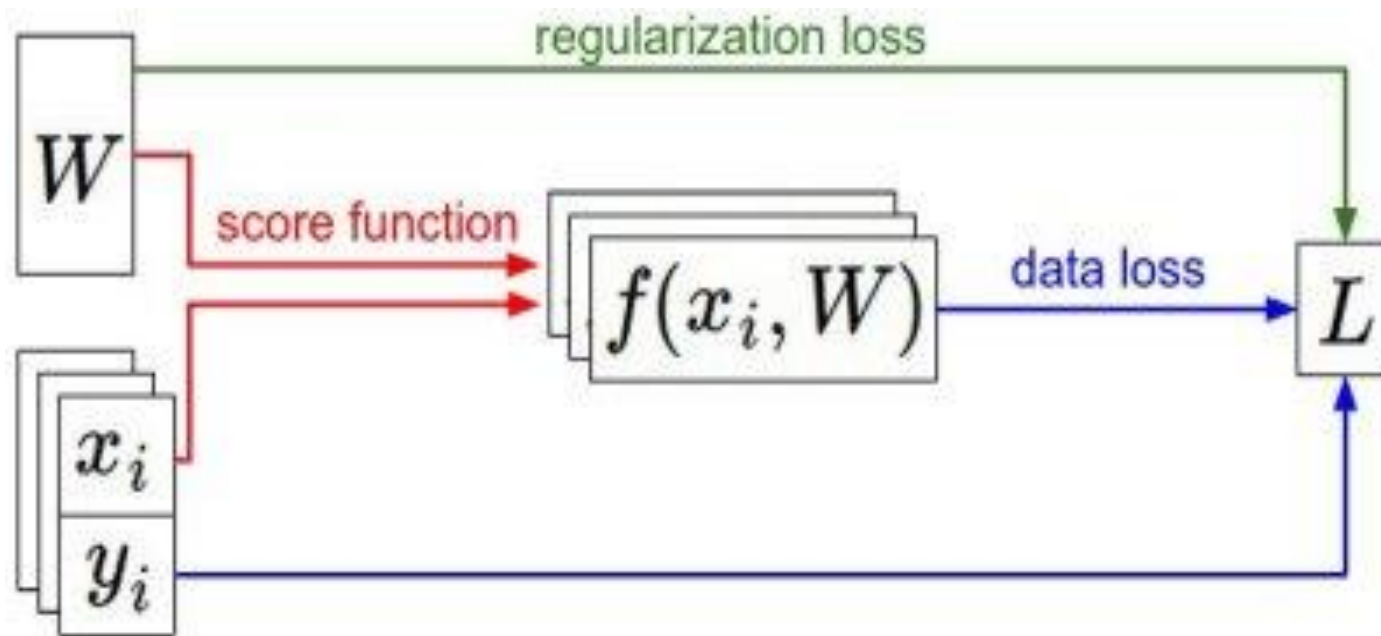
Особенности

1. Определение графа вычислений
2. Расчет графа вычислений
3. Дифференцирование графа

Типы фреймворков

1. [TensorFlow](#) — это комплексная платформа для машинного обучения с открытым исходным кодом. Она была разработана командой Google Brain. Ориентирован на большие проекты.
2. [PyTorch](#) — это среда машинного обучения на языке Python с открытым исходным кодом, обеспечивающая тензорные вычисления с GPU-ускорением. Она была разработана компанией Facebook и представлена в октябре 2016 года, а открыта для сторонних разработчиков — в январе 2017 года. Фреймворк подходит для быстрого прототипирования в исследованиях, а также для любителей и небольших проектов.
3. [Darknet](#) — это фреймворк с открытым исходным кодом, написанный на языке C с использованием программно-аппаратной архитектуры параллельных вычислений CUDA. Он быстрый, лёгкий и удобный в использовании. Также Darknet поддерживает вычисления на базе CPU и GPU.
4. [XGBoost](#) — это фреймворк с открытым исходным кодом, который предлагает систему градиентного бустинга для C++, Java, Python, R, Julia. Он разработан для обеспечения высокой эффективности, гибкости и портативности.

ГРАДИЕНТНЫЕ МЕТОДЫ В МАШИННОМ ОБУЧЕНИИ



ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Введем следующие обозначения:

θ — переменные оптимизации (веса синапсов искусственных нейронов, т.е. W в обозначениях на рисунке)

$\theta \in R^d$, где d — размерность вектора оптимизации.

$J(\theta)$ — целевая функция (функция ошибки или «потери»

$L(x, W, y)$ — в обозначениях на рисунке).

$\nabla_{\theta} J(\theta)$ — градиент целевой функции.

η — коэффициент обучения, определяющий величину шага в направлении локального минимума.

Выделим три основных варианта градиентного спуска.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Batch gradient descent.

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

*Градиентный метод, который рассчитан на вычисление градиента для **всего набора данных** (выборке - *batch*).*

Применение этого метода обладает следующими особенностями:

- 1) Требуется достаточно большого объема памяти.
- 2) Обладает низкой скоростью сходимости.
- 3) Низкой точностью.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Stochastic gradient descent (SGD)

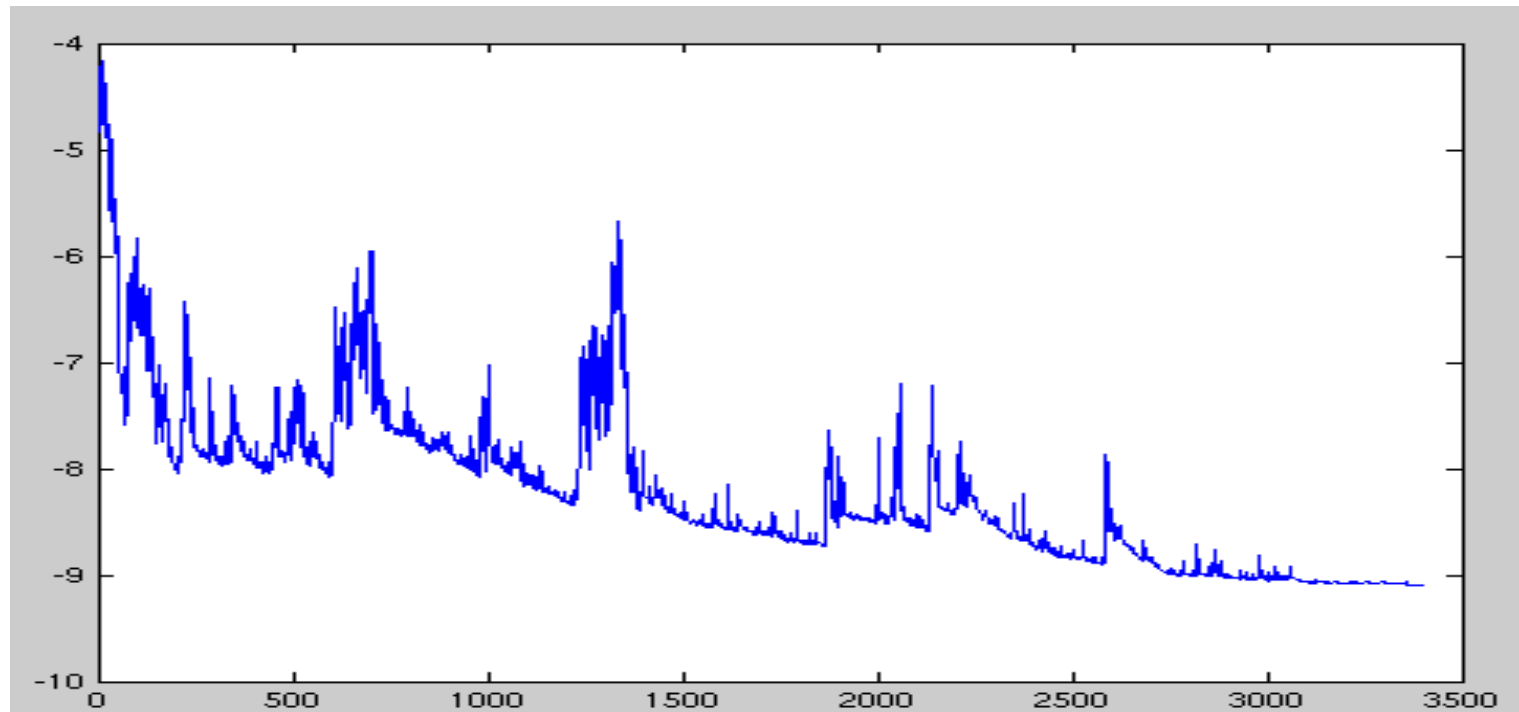
$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$

*В отличие от традиционного градиентного метода, метод стохастического градиента позволяет обрабатывать за одну итерацию относительно небольшой набор обучающих данных из всего набора данных (выборки - *batch*).*

Применение этого метода позволяет избежать недостатки традиционного градиентного метода:

- 1) Требуется значительно меньше необходимого объема памяти.
- 2) Повышается скорость сходимости.
- 3) Увеличивается робастность к невыпуклости целевых функций.
- 4) Возможна оптимизация многоэкстремальных целевых функций.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ



Флуктуации целевой функции в процессе работы алгоритма

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Mini-batch gradient descent

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$$

*За каждую эпоху отбирается n данных ($n = 50 - 250$) небольшой набор обучающих данных из всего набора данных (выборки - *batch*).*

Применение этого метода дает следующие преимущества:

- 1) Уменьшает дисперсию разброса параметров что ведет к более устойчивой сходимости.
- 2) Позволяет использовать матричные и тензорные операции, входящие в состав современных библиотек методов глубокого обучения (смотри слайд 12).

Все это делает использование SGD на минибатчах очень эффективными для решения задач по глубокому обучению.

Далее мы рассмотрим дальнейшее развитие методов SGD.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Momentum

$$v_t = v_{t-1} + \eta \nabla_{\theta} J(\theta)$$
$$\theta = \theta - v_t$$

Этот метод, применительно к SGD позволяет преодолевать овраги. Кроме того, он позволяет увеличить скорость сходимости алгоритмов SGD.

Метод моментов был предложен в 1964 году Б.Т. Поляком.

Другое название этого метода - «метод тяжелого шарика». Он рассматривался нами раньше в лекции посвященной многоэкстремальной оптимизации.

В 1986 году этот метод был использован Джеффри Хинтоном и Дэвидом Руммельхартом для обучения нейронных сетей методом обратного распространения ошибки.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Адаптивные методы

- 1. NAG. (Nesterov accelerated gradient)**
- 2. Adagrad. (Adapted gradient)**
- 3. Adadelat. (Extension of Adagrad)**
- 4. RMSProp. (Adaptive Root Mean Squared)**
- 5. Adam. (Adaptive moment estimation)**
- 6. AdaMax. (Modification Adam)**
- 7. Nadam. (Nesterov modification Adam)**

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

NAG. (Nesterov accelerated gradient)

$$v_t = v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$

$$\theta = \theta - v_t$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adagrad. (Adapted gradient)

$$\begin{aligned}g_{t,i} &= \nabla_{\theta_t} J(\theta_{t,i}) \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \varepsilon}} g_{t,i} \\ G_{t,ii} &= G_{t-1,ii} + g_{t,i}^2\end{aligned}$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adadelta. (Extension of Adagrad)

$$\theta_{t+1} = \theta_t + \frac{\Delta\theta_t}{RMS[\Delta\theta_{t-1}]}$$
$$\Delta\theta_t = - \frac{RMS[\Delta\theta_{t-1}]}{RMS[g_t]} g_t.$$

$$M[\Delta\theta_t^2] = \gamma M[\Delta\theta_{t-1}^2] + (1 - \gamma) \Delta\theta_t^2.$$

$$M[g_t^2] = \gamma M[g_{t-1}^2] + (1 - \gamma) g_t^2.$$

$$RMS[\Delta\theta_t] = \sqrt{M[\Delta\theta_t^2] + \varepsilon}.$$

$$RMS[g_t] = \sqrt{M[g_t^2] + \varepsilon}$$

.

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

RMSProp. (Adaptive Root Mean Squared)

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{M[g_t^2] + \varepsilon}} g_t.$$

$$M[g_t^2] = 0.9M[g_{t-1}^2] + 0.1 g_t^2.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Adam. (Adaptive moment estimation)

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \hat{m}_t \quad .$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\beta_1 = \beta_2 = 0.98.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

AdaMax. (Modification Adam)

$$\theta_{t+1} = \theta_t - \frac{\eta}{u_t} \hat{m}_t.$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$u_t = \max(\beta_2 v_{t-1}, |g_t|)$$

$$\eta = 0.002; \beta_1 = 0.9; \beta_2 = 0.999.$$

ГРАДИЕНТНЫЕ МЕТОДЫ В ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЯХ

Nadam. (Nesterov modification Adam)

Является комбинацией NAG и Adam

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \left(\beta_1 \widehat{m}_t + \frac{(1 - \beta_1) g_t}{1 - \beta_1^t} \right).$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t.$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}.$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\eta = 0.002; \beta_1 = 0.9; \beta_2 = 0.999.$$